Spring MVC (Web Module) - Exercises

Assignment 1: Build a Student Management System (MVC-Based)

Objective:
Implement an end-to-end Spring MVC application (non-REST) using Thymeleaf or JSP, where users can:

- Add, View, Update, Delete students
- Validate user input (e.g., name required, email format)
- Display custom error pages

Requirements:

- Use Spring MVC (DispatcherServlet configuration)
- Create StudentController with CRUD endpoints
- Views using Thymeleaf/JSP
- @Valid on form submission + BindingResult handling
- ExceptionHandler for invalid ID
- Use Spring Boot (or Java config for XML-free setup)

Assignment 2: RESTful Web Service for Product Inventory

Objective:
Design and build a RESTful API for product management.

Endpoints:

- GET /api/products
- GET /api/products/{id}
- POST /api/products (with validation)
- PUT /api/products/{id}
- DELETE /api/products/{id}

Requirements:

- Spring Boot REST controller
- Use @Valid and @Validated for input validation
- Return 400 Bad Request on validation failure
- Global exception handler using @ControllerAdvice
- Return JSON with error message, status, timestamp

Assignment 3: Exception Handling & Input Validation Showcase

Objective:
Create a REST controller that accepts employee details and stores them.

Use cases:

- POST /api/employees with fields: name, email, salary
- Validate: name not blank, email format, salary > 0
- Use @ControllerAdvice to handle:
    - MethodArgumentNotValidException
    - Custom NotFoundException
- Return error response as JSON:
  {
  "timestamp": "...",
  "status": 400,
  "errors": ["Email is invalid", "Salary must be greater than 0"]
  }

Assignment 4: Custom Validator Integration

Objective:
Use a custom validator for validating business logic.

Task:

- Create UserRegistration DTO

    - username, password, confirmPassword, age
- Validate that:
    - password == confirmPassword
    - age ≥ 18
- Use @Valid + custom @PasswordMatches annotation with ConstraintValidator
- Return validation errors as JSON in REST response

Assignment 5: DispatcherServlet Flow Tracing

Objective:
Understand the internal flow of DispatcherServlet and Spring MVC.

Task:

- Create a custom HandlerInterceptor to log:
    - PreHandle: incoming request URL
    - PostHandle: ModelAndView info
    - AfterCompletion: response status
- Add logging in controller methods
- Document and explain flow: client → DispatcherServlet → HandlerMapping → Controller → ViewResolver → Response